

过程类似，都是持久性存储模块。存储函数的创建和存储过程也类似，不同之处是存储函数必须指定返回的类型。

1. 创建存储函数

```
CREATE OR REPLACE FUNCTION 函数名([参数 1 数据类型, 参数 2 数据类型, ...])  
RETURNS <类型>  
AS <过程化 SQL 块>;
```

2. 执行存储函数

```
CALL/SELECT 函数名([参数 1, 参数 2, ...]);
```

3. 修改存储函数

可以使用 ALTER FUNCTION 重命名一个存储函数，格式如下：

```
ALTER FUNCTION 函数名 1 RENAME TO 函数名 2;
```

可以使用 ALTER FUNCTION 重新编译一个存储函数，格式如下：

```
ALTER FUNCTION 函数名 COMPILE;
```

存储函数的概念与存储过程类似，这里就不再赘述了。有关示例可参见本章参考文献[4]。

8.3 JDBC 编程

Java 是一种面向对象的高级编程语言，具有健壮性、安全性、可移植性等优点。考虑到 Java 是最受欢迎的编程语言之一，全球有超过千万的开发者在使用它，本节将介绍如何进行 JDBC 编程，即如何使用 Java 语言调用 JDBC 来进行数据库应用程序的设计。

8.3.1 JDBC 工作原理概述

JDBC 应用系统的体系结构如图 8.5 所示，它由三部分构成：用户应用程序、JDBC 驱动程序管理器和数据源。

1. 用户应用程序

用户应用程序提供用户界面、应用逻辑和事务逻辑。使用 JDBC 开发数据库应用程序时，不用关心底层数据源是什么，只需要知道交互数据源的类型即可。用户应用程序与数据库中数据进行交互时，调用的是 JDBC 提供的标准接口(JDBC APIs)。JDBC 的标准接口主要包括：驱动程序管理接口(java.sql.DriverManager)、请求数据库连接时的连接接口(java.sql.Connection)、向数据源发送 SQL 语句并接收结果的执行接口(java.sql.Statement、java.sql.PreparedStatement、java.sql.CallableStatement)、数据库操作结果集处理接口(java.sql.ResultSet)等。

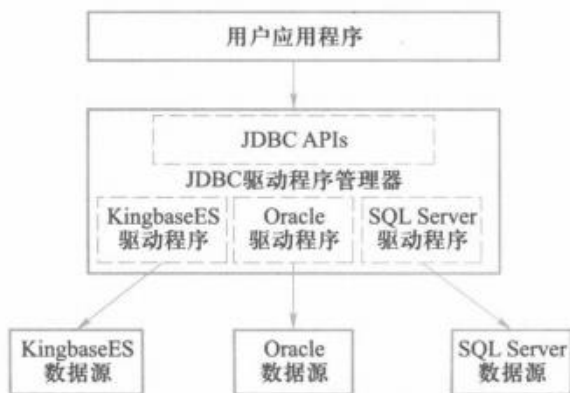


图 8.5 JDBC 应用系统的体系结构

2. JDBC 驱动程序管理器

JDBC 通过驱动程序管理器提供应用系统与数据库平台的独立性。

使用 JDBC 开发应用系统的程序简称为 JDBC 应用程序。JDBC 应用程序与数据库进行交互时不能直接存取数据，其各种操作请求需要通过调用 JDBC 驱动程序管理器提供的标准接口完成。JDBC 驱动程序管理器提供一系列标准接口，各个数据库厂商需要提供 JDBC 驱动程序，建立数据库访问接口与这些标准接口之间的映射。JDBC 应用程序对标准接口访问时，由驱动程序管理器提交给某个关系数据库管理系统的 JDBC 驱动程序，通过调用驱动程序所支持的函数来存取数据库中的数据。数据库的操作结果也通过驱动程序返回给应用程序。如果应用程序要操纵不同数据库管理系统下的数据库，就要动态地链接到不同的驱动程序上。

目前，JDBC 驱动程序主要有本地协议驱动和网络协议驱动两种。本地协议一般是数据源和应用程序在同一台机器上，JDBC 应用程序的接口调用将交由驱动程序直接完成对本地数据文件的存取操作，这时驱动程序相当于数据管理器。网络协议驱动支持客户-服务器、浏览器-服务器等网络环境下的数据访问，这时 JDBC 应用程序的接口调用将交由驱动程序访问网络中的中间件(应用服务器)，该中间件将 JDBC 调用直接或间接转换为特定数据库厂商提供的数据存取访问。

无论使用何种类型的驱动程序，驱动程序将完成数据库访问请求的提交和结果集接收，应用程序使用驱动程序提供的结果集管理接口，操纵执行后的结果数据。

3. 数据源

数据源是用户最终需要访问的数据，包含数据库位置、数据库类型、访问数据库的用户名和密码等信息，实际上是一种数据连接的抽象。应用程序与数据源是通过调用驱动管理器 DriverManager 的静态方法 getConnection(URL, UserName, Password) 建立连接的，参数分别为数据库的 URL、数据库用户名及密码。

8.3.2 JDBC APIs 基础

JDBC APIs 是各个数据库厂商提供的 JDBC 应用程序开发接口都要遵循的标准。

1. JDBC 中的常用类

使用 JDBC 进行应用程序开发涉及的所有类都包含在 `java.sql` 包中, 常用的有:

- ① 驱动程序类(`java.sql.Driver`), 由各数据库厂商提供。
- ② 驱动程序管理类(`java.sql.DriverManager`), 作用于应用程序与驱动程序之间。
- ③ 数据库连接类(`java.sql.Connection`), 用于建立与指定数据库的连接。
- ④ 静态 SQL 语句执行类(`java.sql.Statement`), 用于执行静态 SQL 语句并返回结果。
- ⑤ 动态 SQL 语句执行类(`java.sql.PreparedStatement`), 用于执行含参 SQL 语句并返回结果。

⑥ 存储过程语句执行类(`java.sql.CallableStatement`), 用于执行存储过程语句并返回结果。

⑦ 结果集处理类(`java.sql.ResultSet`), 用于检索结果集中的数据。

注意: JDBC 应用程序开发接口包含在上述 Java 类中, 不同 JDBC 版本上的接口名与接口的使用可能是有差异的, 读者必须注意使用的版本。

2. 数据类型

JDBC 定义了两套数据类型: SQL 数据类型和 Java 数据类型。SQL 数据类型用于数据源, 涉及例如创建关系模式时指定属性的数据类型。Java 数据类型用于应用程序, 涉及例如程序中定义的变量的数据类型。它们之间的转换规则如表 8.4 所示。当用户的应用程序通过 JDBC APIs 从数据库中存取数据时, Java 程序所使用的变量类型需要与数据库中关系模式属性的类型建立映射关系。例如, 通过结果集处理对象中的 `get` 方法从数据库中获得结果集, 并把记录中的属性值赋给应用程序变量; 为动态 SQL 语句执行对象绑定应用程序变量。

表 8.4 SQL 数据类型与 Java 数据类型之间的转换规则

数据类型	SQL 数据类型	Java 数据类型
SQL 数据类型	数据源之间转换	应用程序变量传递给语句参数
Java 数据类型	从数据库中读取数据赋值给应用程序变量	应用程序变量之间转换

8.3.3 使用 JDBC 操纵数据库的工作流程

使用 JDBC 操纵数据库的工作流程如图 8.6 所示。本小节将结合具体的应用实例来介绍如何使用 JDBC 开发应用系统。



图 8.6 使用 JDBC 操纵数据库的工作流程

1. 加载驱动程序

在连接数据库之前，首先加载要连接数据库的驱动程序。可以使用 `java.lang.Class` 类的静态方法 `forName(String className)` 来加载指定数据库的驱动程序。一般来讲，在数据库厂商提供的 `Driver` 类中（例如 Kingbase 数据库的 `com.kingbase.Driver`），其静态代码段会默认包含一次驱动程序的注册操作。`forName` 方法在加载类时会自动执行一次该静态代码段，从而向驱动程序管理器注册一个驱动程序。

[例 8.7] 对 Kingbase、Oracle、SQL Server 数据库加载驱动。

```
Class.forName("com.kingbase.Driver");           /* Kingbase */
Class.forName("oracle.jdbc.OracleDriver");      /* Oracle */
Class.forName("com.microsoft.jdbc.sqlserver.SQLServerDriver"); /* SQL Server */
```

注意：在 JDBC 4.0 及以后版本不再需要使用 `Class.forName()` 显式地加载 JDBC 驱动程序。此外，根据 JDBC 开发规范，连接数据库等操作需要进行异常处理。本书为了叙述方便省略了异常处理，在实际的应用开发中建议参考 JDBC 开发规范补充异常处理逻辑，以增强应用系统的健壮性。

2. 建立与数据库的连接

(1) 定义连接的 URL

加载数据库驱动程序后，可以与数据库建立连接，连接需要通过 URL 地址定义。URL 地

址包含连接数据库所需的协议、子协议和数据库名称, 定义格式为

<协议名>:<子协议名>:<数据库名称>

JDBC 连接中的协议名总是 jdbc, 子协议名和数据库名称由具体连接的 DBMS 决定。

[例 8.8] 定义与 Kingbase、Oracle、SQL Server 数据库连接的 URL。

```
strURL="jdbc:kingbase://"+服务器地址+": "+端口号+"/"+数据库名;  
strURL="jdbc:oracle:thin:@ "+服务器地址+": "+端口号+": "+数据库名;  
strURL="jdbc:microsoft:sqlserver://"+服务器地址+": "+端口号+": "+数据库名;
```

Kingbase、Oracle、SQL Server 的默认端口号分别为 54321、1521、1433。

(2) 建立连接

利用生成的 URL 建立与数据库的连接。连接数据库时, 需要向 java.sql.DriverManager 类请求并创建一个数据库连接类(Connection)对象。使用 java.sql.DriverManager 类提供的静态方法 getConnection, 传入指定的连接 URL、数据库用户名和密码建立连接。

[例 8.9] 建立与 Kingbase 数据库的连接, 假定 Kingbase 服务器地址为 192.168.0.118, 端口号为 54321, 数据库名为 DB-Student, 用户名为 Info001, 密码为 123456。

```
String strURL="jdbc:kingbase://192.168.0.118:54321/DB-Student";  
Connection conn=DriverManager.getConnection(strURL,"Info001","123456");
```

3. 执行 SQL 语句

(1) 创建语句执行类对象

与数据库建立连接后, 就可以开始操作数据库了。JDBC 提供了语句执行类对象用来执行用户提交的 SQL 语句, 并接收 SQL 查询的结果集。

利用数据库连接类的 createStatement 方法可以创建静态语句执行类对象(Statement), 静态语句执行类对象只能用于执行静态的 SQL 语句。静态语句执行类还派生出两个动态语句执行类(PreparedStatement)和存储过程执行类(CallableStatement), 它们可以用于执行更高级的 SQL 语句。

动态语句执行类对象用来执行动态的 SQL 语句, 经过预编译, 可以有效防止 SQL 注入攻击, 从而提高数据库的安全性。另外, 预编译结果可以存储在动态语句执行类对象中, 多次执行该 SQL 语句时可以提高运行效率。

JDBC 还提供一个存储过程执行类, 专门用于执行数据库存储过程。

(2) 执行 SQL 语句

SQL 语句可以通过以下三种方法执行: executeQuery、executeUpdate 和 execute。

① executeQuery 用来执行数据库查询语句, 成功执行后返回一个 ResultSet 类(结果集)对象。

② executeUpdate 用来处理增加、删除、修改以及数据库定义语句, 成功执行后返回一个

int 类型值。

③ execute 用来处理存储过程或动态 SQL 语句, 成功执行后返回一个 boolean 类型值。

[例 8.10] 使用 JDBC 向课程评价表中插入一条记录。课程评价关系模式为 ClassAssess(Sno, Tno, TCno, Assess, CAtype Feedback)。

```
/* 创建 PreparedStatement 类对象 */
PreparedStatement stmt=conn. prepareStatement("INSERT INTO ClassAssess VALUES(?,?,?,?,?,?)");
/* 生成 PreparedStatement 类对象的动态参数,注意第六个字段 Feedback 未设置输入值 */
stmt.setString(1, "20180001");           /* 设置学生学号 */
stmt.setString(2, "19950018");           /* 设置职工号 */
stmt.setString(3, "81001-01");           /* 设置教学班号 */
stmt.setString(4, "老师讲得很出色");     /* 设置学生评价意见 */
stmt.setBoolean(5, true);                 /* 设置学生评价意见类型 */
stmt.executeUpdate();
```

[例 8.11] 利用例 8.5 创建的 compGPA 存储过程, 使用 JDBC 查询学号为 20180001 同学的平均学分绩点。

```
CallableStatement stmt=conn. prepareCall("{CALL compGPA (?,?,)}");
/* 创建存储过程执行类对象 */
/* 生成 CallableStatement 类对象中的动态参数 */
stmt.setString(1, "20180001");           /* 设置输入参数 */
stmt.registerOutParameter(2, Types. REAL); /* 设置输出参数 */
stmt.execute();                           /* 执行存储过程 */
float fGPA=stmt. getFloat(2);              /* 获取输出参数 fGPA */
```

4. 处理结果集

使用 executeQuery 方法执行数据库查询语句, 成功执行后返回一个结果集类对象(ResultSet)。通过遍历结果集类对象中的每一个元组, 执行用户定义的应用逻辑。此外, 通过 ResultSet 类提供的 getXXX() 方法来获取结果集中元组的属性值。XXX 代表某种数据类型, 当用户想要获取某种数据类型的某列值时, 可以指定参数为列号(JDBC 的列从 1 开始)或列名。

JDBC 使用游标来处理结果集数据, 游标可分为三种类型: TYPE_FORWARD_ONLY 游标、TYPE_SCROLL_INSENSITIVE 游标和 TYPE_SCROLL_SENSITIVE 游标。TYPE_FORWARD_ONLY 游标只能在结果集中向下滚动, 它是 JDBC 的默认游标类型。TYPE_SCROLL_INSENSITIVE 游标和 TYPE_SCROLL_SENSITIVE 游标是双向滚动游标, 区别就在于前者不会同步更新数据库中的操作, 后者会在结果集类对象中及时跟踪数据库的更新操作。这个参数可以通过指定结果集类对象的 resultSetType 参数进行修改。

JDBC 游标的打开方式不同于嵌入式 SQL, 不是显式声明而是由系统自动产生, 当结果集

刚刚生成时, 游标指向第一行数据之前, 通过 `next()`、`previous()` 等操作来移动游标获取结果集中的每一行数据。

[例 8.12] 打印教师职工号为 19950018, 教学班号为 81001-01 的学生课程评价详情。

```
String SQL "SELECT Sno, Assess, CAtype, Feedback FROM ClassAssess
WHERE Tno='19950018' AND TCno = '81001-01';
ResultSet rs = stmt.executeQuery(SQL);
while(rs.next()) {
    String Sno = rs.getString("Sno");           /* 等价于 rs.getString(1) */
    String strAssess = rs.getString("Assess");    /* 等价于 rs.getString(2) */
    Boolean bCAtype = rs.getBoolean("CAtype");   /* 等价于 rs.getBoolean(3) */
    String strFeedback = rs.getString("Feedback"); /* 等价于 rs.getString(4) */
    /* 打印课程评价详情, 注意隐藏学生的信息, 例如学生的学号 */
    System.out.printf("[ %s,%b,%s]%n", strAssess, bCAtype, strFeedback);
}
```

5. 释放资源

处理结束后, 应用程序将首先关闭结果集, 再关闭语句执行类对象, 最后释放数据库连接并与数据库服务器断开。关闭结果集使用结果集类对象提供的 `close` 方法。类似地, 关闭语句执行类对象和释放数据库连接也分别使用执行语句类对象和连接类对象的 `close` 方法。

[例 8.13] 基于 JDBC 实现 8.1.1 小节的任务 4。

为了简化描述, 输出字符串 `str` 到网页的代码使用 `System.out.println(str)` 函数代替。

如图 8.7 所示, 任务 4 涉及的实体型包括“教师”“教学班”“学生”, 实体型之间的联系包括“教师”与“教学班”之间的“讲授”, “学生”与“教学班”之间的“选课”, “教学班”“教师”“学生”三者之间的“课程评价”。

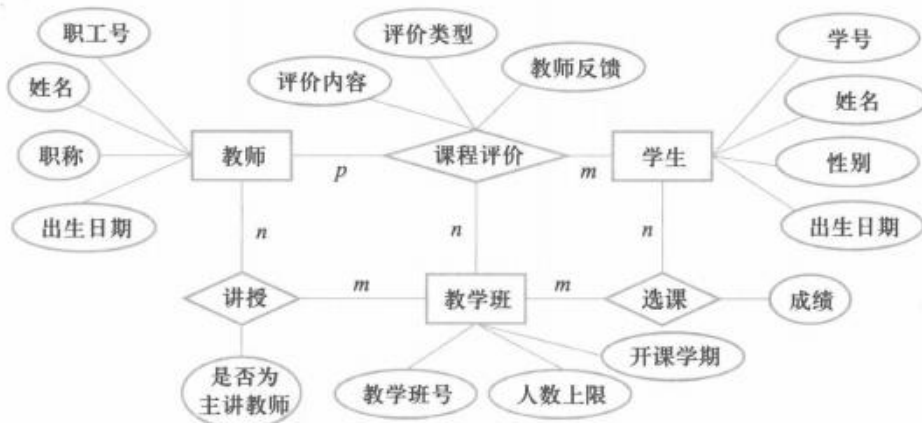


图 8.7 任务 4 涉及的实体型及其之间的联系

其中,“选课”关系模式为 SC(Sno,TCno,Grade),分别表示学生学号、教学班号、成绩。“讲授”关系模式为 Teaching(TCno,Tno,IsLeading),分别表示教学班号、职工号、是否为主讲教师。“课程评价”关系模式为 ClassAssess(Sno,Tno,TCno,Assess,CAtype,Feedback),分别表示学生学号、职工号、教学班号评价内容、评价类型、教师反馈。此外,“教师”“教学班”关系模式分别命名为 Teacher、TeachingClass。

① 数据库连接的管理。

```
public class ConnectionManager{
    static final String jdbcUrl = "jdbc:kingbase:// 192.168.0.118:54321/DB-Student";
    static final String jdbcUsername = "Info001";
    static final String jdbcPassword = "123456";
    private static Connection connection = null;
    /* 静态代码段,加载 Kingbase 数据库驱动程序 */
    static{
        Class.forName("com.kingbase.Driver");
    }
    public static Connection createConnection(){
        connection = DriverManager.getConnection(jdbcUrl,jdbcUsername,jdbcPassword);
        System.out.println("数据库连接成功...");
        return connection;
    }
    /* 释放资源 */
    public static void release(){
        if (connection != null) connection.close();
        System.out.println("释放资源成功...");
    }
}
```

② 学生浏览指定的教学班和授课教师,并输入课程评价。

```
Connection conn = ConnectionManager.createConnection();
Statement stmtnt = conn.createStatement();
/* 设置如下任务的 SQL 语句:获取学生所选的教学班及授课教师,*/
/* 假设 strSno 为该学生学号 */
String SQL4ClassAssess = "SELECT TCno,
    Tno FROM Student,Teacher,Teaching, SC
    WHERE Teacher.Tno=Teaching.Tno AND Teaching.TCno=SC.TCno AND SC.Sno="+strSno;
/* 获取执行语句 SQL4ClassAssess 的结果集 */
ResultSet rs = stmtnt.executeQuery(SQL4ClassAssess);
```



```

/* 设置插入学生课程评价的动态 SQL 语句 */
String insertSQL = "INSERT INTO ClassAssess( Sno, Tno,
    TCno, Assess, CAtype) VALUES(?,?,?,?,?)";
PreparedStatement ps = conn.prepareStatement(insertSQL);
while(rs.next()) {
    /* 阅读该学生所选的教学班及授课教师 */
    String strTno = rs.getString("Tno");
    String strTCno = rs.getString("TeachingClassNo");
    /* 对该授课教师讲授的教学班进行评价 */
    ps.setString(1, strSno);
    ps.setString(2, strTno);
    ps.setString(3, strTCno);
    ps.setString(4, "老师讲得很出色");
    /* 课程评价根据用户输入进行设置,这里仅仅是示例 */
    /* 1 为正面评价,0 为负面评价;评价类型可由用户设置 */
    ps.setBoolean(5, true);
    ps.executeUpdate();
}
rs.close();
ps.close();
stmt.close();
ConnectionManager.release();

```

③ 教师浏览教学班学生的评价意见,并针对每条评价逐一做出回复。

```

Connection conn=ConnectionManager.createConnection();
Statement stmt=conn.createStatement();
/* 获取教师指定教学班的学生课程评价。假设教师职工号为 strTno, */
/* 教学班号为 strTCno */
ResultSet rs = stmt.executeQuery("SELECT Sno, Assess, CAtype,
    Feedback FROM ClassAssess WHERE TCno
    =" + strTCno + " AND Tno
    =" + strTno);
/* 设置更新教师课程意见反馈的动态 SQL 语句 */
String updateSQL = "UPDATE ClassAssess SET Feedback
    =? WHERE Sno
    =? AND Tno
    =? AND TCno
    =?";

```

```
PreparedStatement ps = conn.prepareStatement(updateSQL);
while (rs.next()) {
    /* 阅读每一条课程评价 */
    String strSno = rs.getString("Sno");
    String strAssess = rs.getString("Assess");
    Boolean bCAtype = rs.getString("CAtype");
    String strFeedback = rs.getString("Feedback");
    /* 打印课程评价详情,注意隐藏学生的信息,例如学生的学号 */
    System.out.printf("[%s,%b,%s]%n",strAssess, bCAtype, strFeedback);
    /* 对该评价进行回复 */
    ps.setString(1, "感谢你的评价");          /* 反馈意见根据教师的输入进行设置 */
    ps.setString(2, strSno);                    /* 设置学生学号 */
    ps.setString(3, strTno);                    /* 设置教师职工号 */
    ps.setString(4, strTCno);                   /* 设置教学班号 */
    ps.executeUpdate();                        /* 更新教师课程意见反馈 */
}
rs.close();
ps.close();
stmt.close();
ConnectionManager.release();
```

从例 8.13 可以看出,在应用程序中直接使用 JDBC 操纵数据库,需要经过图 8.6 所示的加载驱动程序—建立与数据库的连接—执行 SQL 语句—处理结果集—释放资源的整个过程(加载驱动程序可以省略)。程序的代码管理显得比较复杂。此外,在程序的模块中需要获取数据库连接,在数据操作完毕之后再关闭数据库连接。这样对数据库进行频繁连接、开启和关闭操作,会造成数据库资源的浪费,影响数据库的性能。8.4 节基于 MVC 框架的数据库应用开发可以克服这些缺点。

* 8.4 基于 MVC 框架的数据库应用开发

MVC(model-view-controller, 模型-视图-控制器模式)是一种使用业务模型(Model)、用户结果呈现(View)以及业务模型控制器(Controller)来进行 Web 数据库应用开发的编程框架。基于 MVC 框架进行 Web 数据库应用开发,其核心思想在于功能的解耦以及功能的模块化。以本书第 7 章中介绍的“高校本科教务管理”信息系统的子系统“教师教学管理”为例进行介绍(参见图 7.30),该子系统共包括“教师”“教室”“时间片”“教学班”等实体,以及“排课”“讲授”“课程评价”等联系。其功能包括: